Vole Machine PlantUML Model

https://gemini.google.com/app/b81ffe42f259d262

User prompt: write in plantuml a vole machine domain that can access memory and do some instructions and have menu to choose what i want it to do

Response: Here is a PlantUML class diagram representing the **Vole Machine** domain model (based on the classic pedagogical computer model from *Computer Science Illuminated*). It includes components for memory access, instruction execution, and a menu interface to control the machine.

PlantUML Code

```
@startuml
skinparam classAttributeIconSize 0
skinparam backgroundColor #F9F9F9

class VoleMachine {
    - pc: int
    - ir: int
    - registers: int[16]
    - isRunning: boolean
    + powerOn(): void
    + powerOff(): void
    + runProgram(): void
    + executeStep(): void
}

class Memory {
    - size: int
    - cells: byte[256]
    + read(address: int): byte
    + write(address: int, value: byte): void
    + loadProgram(program: byte[]): void
    + dumpMemory(): void
}

class ControlUnit {
    + fetch(pc: int, memory: Memory): int
    + decode(instruction: int): void
}

class ALU {
    + executeALUOperation(opcode: int, r: int, val1: int, val2: int): int
}

class MenuUI {
    + displayMenu(): void
    + getChoice(): int
    + handleChoice(choice: int): void
    + displayStatus(machine: VoleMachine, memory: Memory): void
}
```

```
' Relationships
VoleMachine "1" *-- "1" Memory : manages >
VoleMachine "1" *-- "1" ControlUnit : uses >
VoleMachine "1" *-- "1" ALU : uses >
MenuUI ..> VoleMachine : controls >
MenuUI ..> Memory : inspects/edits >

@enduml
```

Domain Model Breakdown

- **`MenuUI`**: Handles the user interaction, presenting a menu to choose actions (e.g., loading a program, stepping through instructions, viewing memory contents, or resetting the machine).
- **`VoleMachine`**: The core computer model that maintains the state, including the Program Counter (`pc`), Instruction Register (`ir`), and the 16 general-purpose registers (`registers`).
- **`Memory`**: Represents the addressable storage (typically 256 bytes in the standard Vole model) allowing reading, writing, and program loading.
- **`ControlUnit`**: Responsible for fetching instructions from memory using the program counter and decoding them.
- **`ALU` (Arithmetic Logic Unit)**: Performs the arithmetic and logical operations required by the machine's instruction set architecture (ISA).

Would you like to expand this into a sequence diagram to show how the menu interacts with the memory and instruction cycle?
